robbit-esp System Manual

Kise Laboratory
Created: September 2025

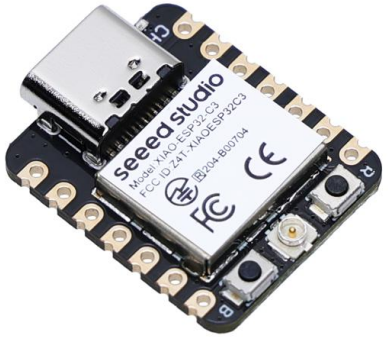
Overview

- This manual describes the hardware and software of robbit-esp.

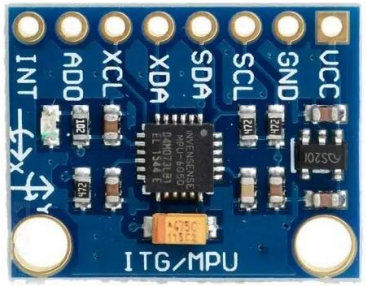
- This manual helps readers understand the robbit-esp system and improve its performance.

robbit-esp: Hardware

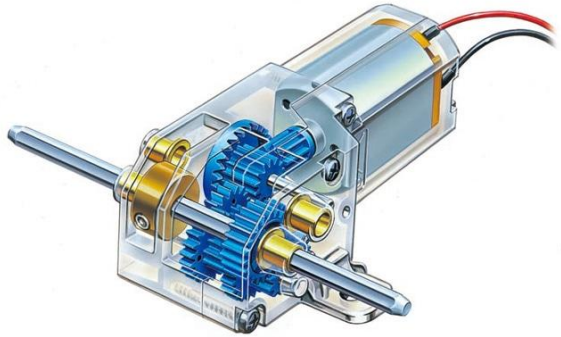
- Microcontroller board: XIAO ESP32-C3
- Sensor: MPU-6050 (IMU)
  - Accelerometer and angular-velocity (gyroscope) sensor
- Geared motor: Mini Motor Standard Gearbox (TAMIYA)



XIAO ESP32-C3



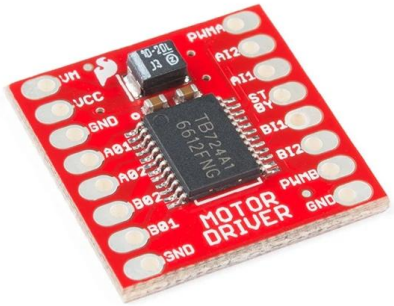
MPU-6050



Mini Motor Standard Gearbox (TAMIYA)

robbit-esp: Hardware

- Motor driver: TB6612FNG motor-driver module
- Tires: Slim Tire Set (55 mm diameter), 70193
- Battery: EEMB 653042 (3.7 V)



TB6612FNG

Motor Driver

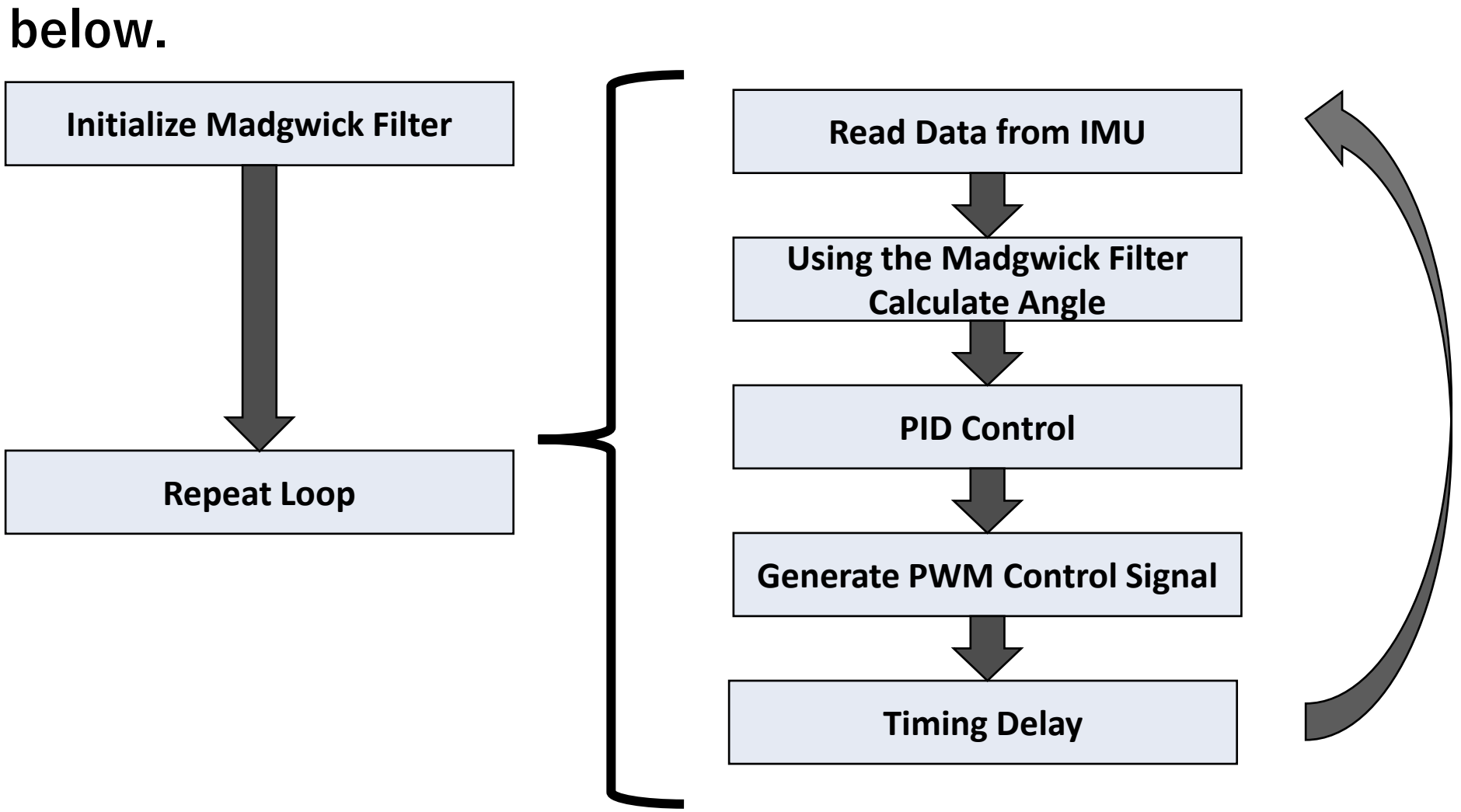
Slim Tire Set

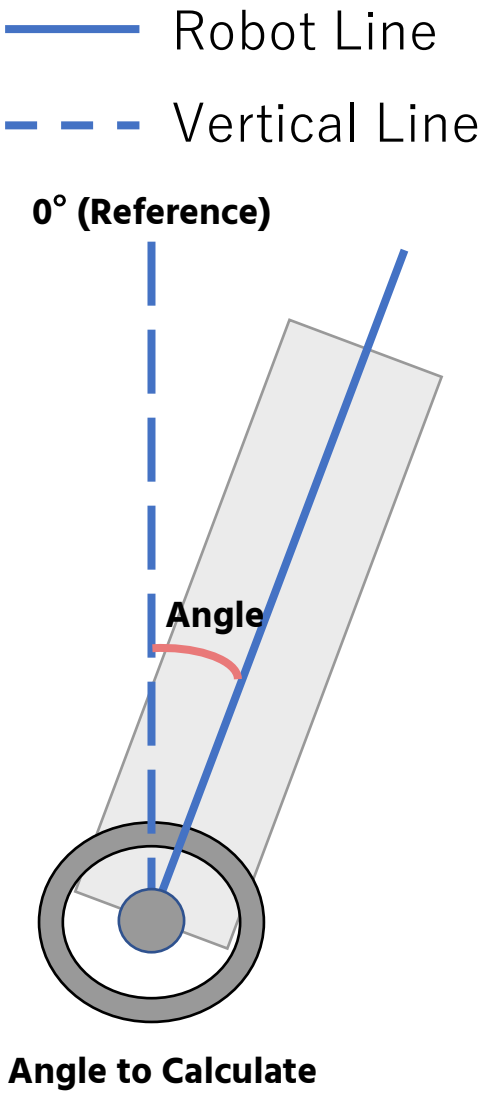
EEMB 653042

Rechargeable Battery

Software: Overview

The software is implemented in robbit-esp.ino. The processing flow is shown below.





Software: Reading Data from the IMU

Institute of SCIENCE TOKYO

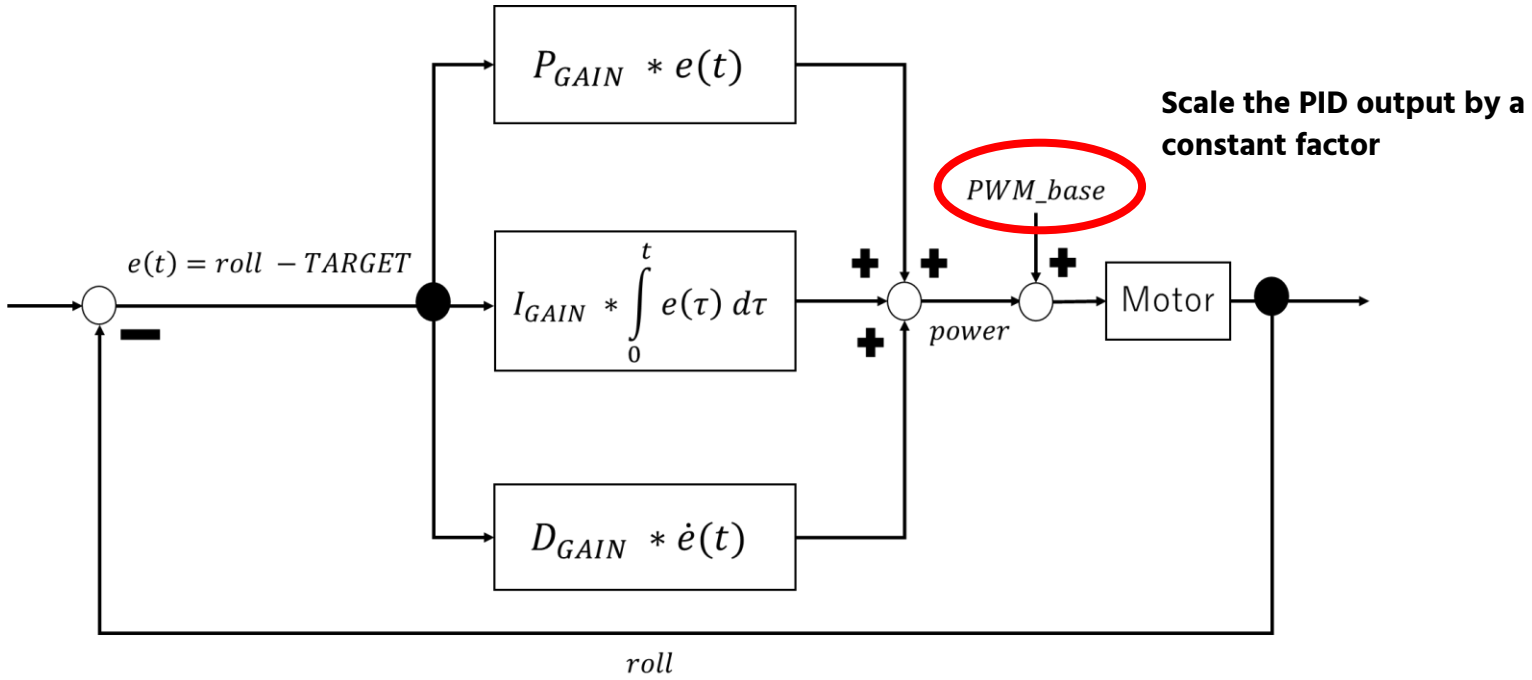
- Read three-axis acceleration and angular velocity from the IMU.
- Communication with the IMU uses an existing library.

- The angle is calculated from three-axis acceleration and angular velocity using a Madgwick filter.
- A gain-adjustment function is added to the Madgwick-filter library.

```
mpu.getMotion6(&ay, &ax, &az, &gy, &gx, &gz);
MadgwickF.updateIMU(gz/131.0, gy/131.0, gx/131.0, az/16384.0, ay/16384.0, ax/16384.0);
roll = MadgwickF.getRoll() - 90.0; // Note
```

Software: PID Control

- PID Control
- The proportional, integral, and derivative terms are combined to determine the control output (angle).
- The influence of each term is determined by its gain (P_GAIN, I_GAIN, D_GAIN).
- When the deviation is small, the motor may not rotate, so an offset called PWM_base is added.



robbit-esp PID Control Block Diagram

PID Control: Proportional Term (P)

Institute of SCIENCE TOKYO

- The proportional term represents the error between the control output and the target value.
- Increasing P_GAIN makes the system respond more sensitively to deviations.

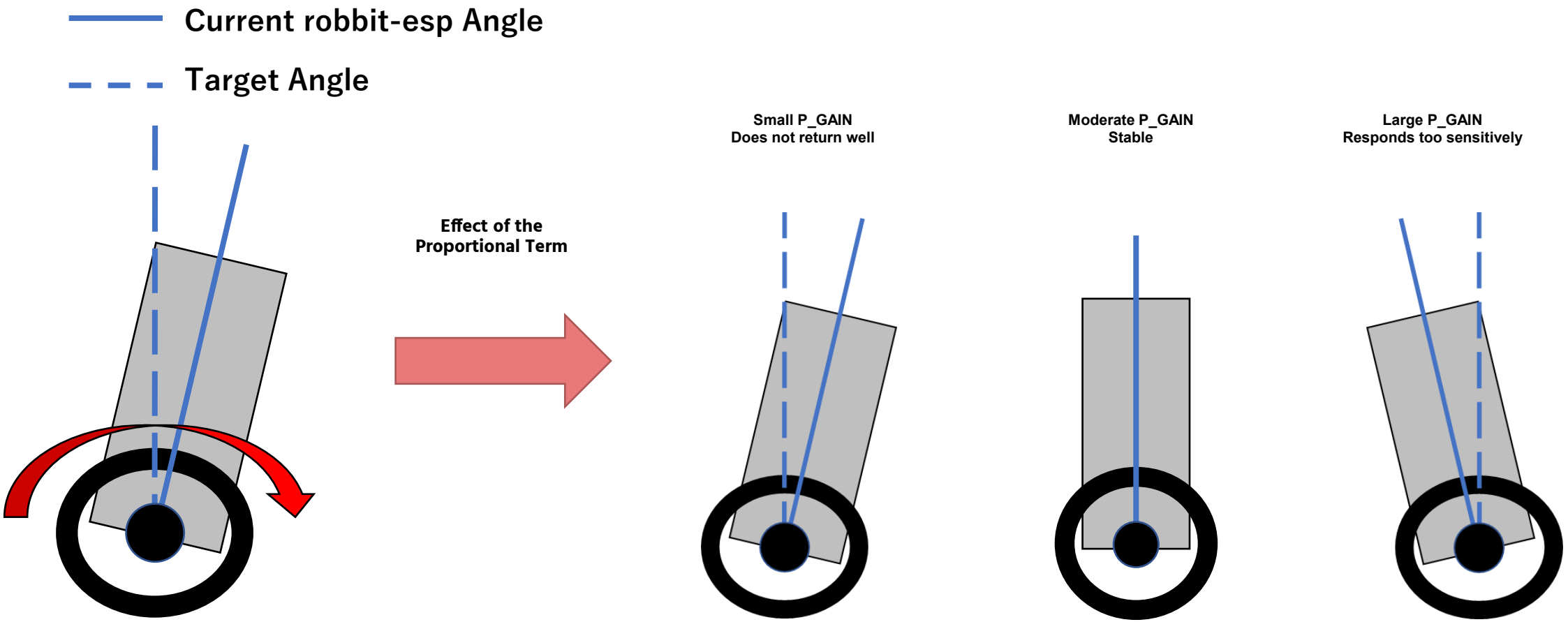


Illustration of the Proportional Term's Effect

PID Control: Integral Term (I)

■ The integral term integrates the error between the control output and the target value over time.
■ Increasing I_GAIN can reduce the deviation, but if it is too large, the accumulated integral term becomes excessive, degrading responsiveness and stability.

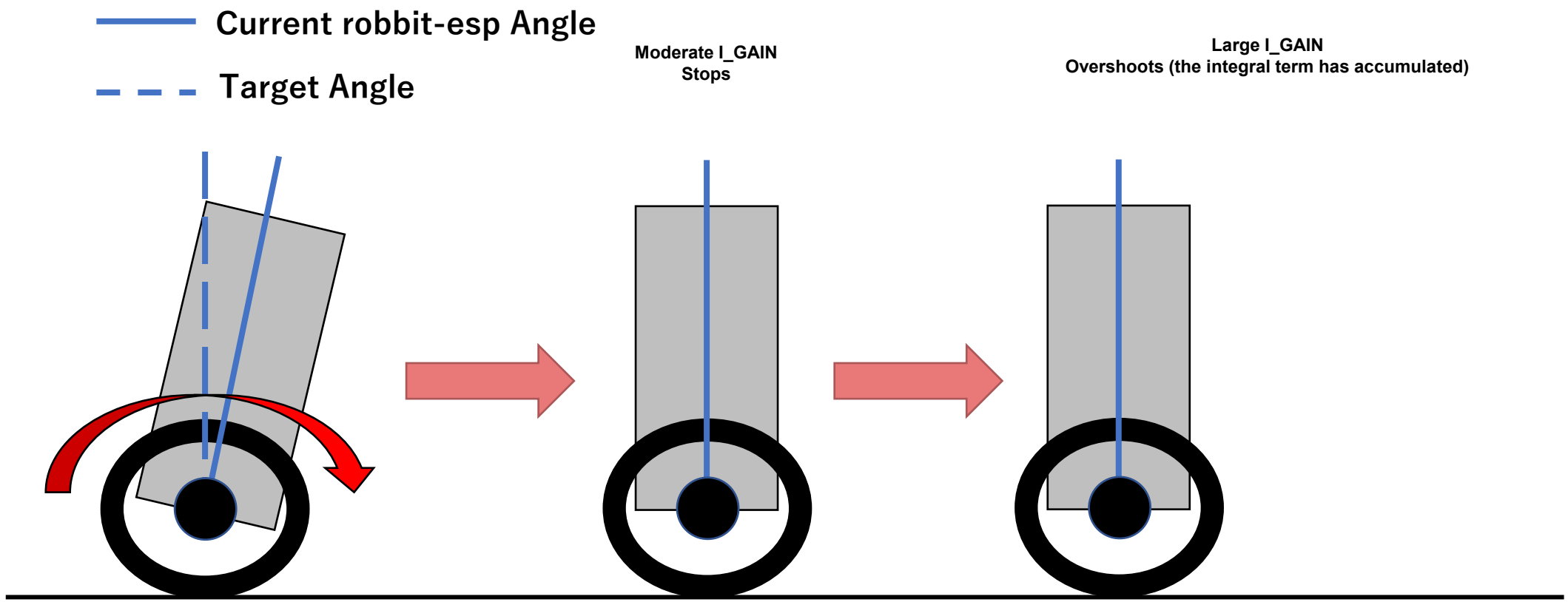


Illustration of the Integral Term's Effect

PID Control: Anti-Windup

Institute of SCIENCE TOKYO

■ Anti-windup is used to suppress the accumulation of the integral term described on the previous page.
■ One implementation method is to set upper and lower limits for the calculated integral term, and update the integral term only when it is within the specified range.

Update the value of the integral term

$$I = \begin{cases} I_{MAX} & (|I + P * dt| > I\_MAX) \\ -I_{MAX} & (|I + P * dt| > I\_MAX) \\ I + P * dt & else \end{cases}$$

PID Control: Derivative Term (D)

Institute of SCIENCE TOKYO

- The derivative term differentiates the error between the control output and the target value over time.
- Increasing D_GAIN suppresses overshoot beyond the target angle, but if it is too large, it takes longer to reach the target angle.

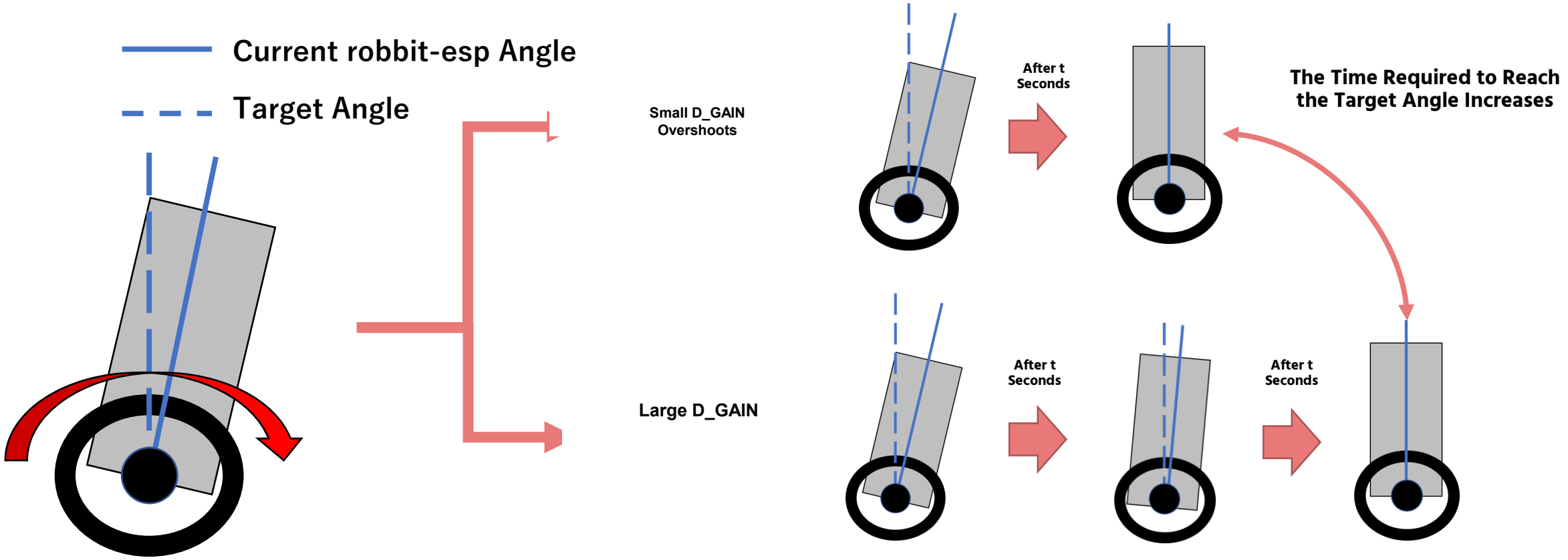


Illustration of the Derivative Term's Effect

Software: PID Control

Institute of SCIENCE TOKYO

The PID controller is implemented as follows.

```
// PID control
P = (parameter.target - roll) / 90.0;          Calculate Proportional Term
if(fabsf(I + P * dt) < I_MAX)  I += P * dt;  // cap    Calculate Integral Term (Anti-Windup)
D = (P - preP) / dt;      Calculate Derivative Term
preP = P;

power = parameter.Kp * P + parameter.Ki * I + parameter.Kd * D;  Calculate Output
```

- P is the error between the target angle (TARGET) and current angle (roll).
- I integrates the proportional error using a rectangular approximation.

The `if (fabsf(I + P * dt))` expression implements the anti-windup condition.

- D is calculated using the finite-difference derivative formula.
- Finally, the terms are combined in power to calculate the output.

Software: Determining the PWM Signal

Institute of SCIENCE TOKYO

The ratio of the waveform that remains HIGH in PWM control is represented using PWM_base and V_max, as shown below.

power represents the calculation result of PID control, and pwm_base represents the offset added to the PWM signal.

$$\mathrm{PWM} = \begin{cases} \mathrm{power} + \mathrm{PWM\_base} & (|\mathrm{power}| < \mathrm{V\_MAX}) \\ \mathrm{V\_MAX} & (|\mathrm{power}| \geq \mathrm{V\_MAX}) \end{cases}$$

In addition, the sign of power is used as a control signal to determine the rotation direction of the motor.

Software: BLE Communication

Institute of SCIENCE TOKYO

- An existing library provides the BLE connection between the PC and robbit-esp.
- Callback functions asynchronously send and receive data when the PC issues read or write requests.

```
class MyCallbacks: public BLECharacteristicCallbacks {
  void onWrite(BLECharacteristic *pCharacteristic) {
    String status = pCharacteristic->getValue();
    int spaceindex = status.indexOf(' '); //index of space

    String parameter = status.substring(0, spaceindex); //parameter name
    float value = status.substring(spaceindex, status.length()).toFloat();

    if (parameter == "1") {
      target = value;
    } else if(parameter == "2"){
      Kp = value;
    } else if(parameter == "3"){
      Ki = value;
    } else if(parameter == "4"){
      Kd = value;
    } else if(parameter == "5"){
      pwm_base = value;
    } else if(parameter == "6"){
      vmax = value;
    }
  }

  void onRead(BLECharacteristic *pCharacteristic) {

  }
```

Callback Functions

- **onWrite: Handles write requests from the PC**
- **onRead: Handles read requests from the PC**

  In this program, the loop function sets the value sent to the PC, so onRead performs no additional processing.